

CORE CONCEPTS

Work with sessions

How sessions persist agent conversation history, and when to use continue, resume, and fork to return to a prior run.

A session is the conversation history the SDK accumulates while your agent works. It contains your prompt, every tool call the agent made, every tool result, and every response. The SDK writes it to disk automatically so you can return to it later.

Returning to a session means the agent has full context from before: files it already read, analysis it already performed, decisions it already made. You can ask a follow-up question, recover from an interruption, or branch off to try a different approach.

ⓘ Sessions persist the **conversation**, not the filesystem. To snapshot and revert file changes the agent made, use [file checkpointing](#).

This guide covers how to pick the right approach for your app, the SDK interfaces that track sessions automatically, how to capture session IDs and use `resume` and `fork` manually, and what to know about resuming sessions across hosts.

Choose an approach

How much session handling you need depends on your application's shape. Session management comes into play when you send multiple prompts that should share context. Within a single `query()` call, the agent already takes as many turns as it needs, and permission prompts and `AskUserQuestion` are **handled in-loop** (they don't end the call).

What you're building	What to use
One-shot task: single prompt, no follow-up	Nothing extra. One <code>query()</code> call handles it.
Multi-turn chat in one process	<code>ClaudeSDKClient</code> (Python) or <code>continue: true</code> (TypeScript). The SDK tracks the session for you with no ID handling.
Pick up where you left off after a process	<code>continue_conversation=True</code> (Python) / <code>continue: true</code>

What you're building	What to use
restart	(TypeScript). Resumes the most recent session in the directory, no ID needed.
Resume a specific past session (not the most recent)	Capture the session ID and pass it to <code>resume</code> .
Try an alternative approach without losing the original	Fork the session.
Stateless task, don't want anything written to disk (TypeScript only)	Set <code>persistSession: false</code> . The session exists only in memory for the duration of the call. Python always persists to disk.

Continue, resume, and fork

Continue, resume, and fork are option fields you set on `query()` (`ClaudeAgentOptions` in Python, `Options` in TypeScript).

Continue and **resume** both pick up an existing session and add to it. The difference is how they find that session:

- **Continue** finds the most recent session in the current directory. You don't track anything. Works well when your app runs one conversation at a time.
- **Resume** takes a specific session ID. You track the ID. Required when you have multiple sessions (for example, one per user in a multi-user app) or want to return to one that isn't the most recent.

Fork is different: it creates a new session that starts with a copy of the original's history. The original stays unchanged. Use fork to try a different direction while keeping the option to go back.

Automatic session management

Both SDKs offer an interface that tracks session state for you across calls, so you don't pass IDs around manually. Use these for multi-turn conversations within a single process.

Python: `ClaudeSDKClient`

`ClaudeSDKClient` handles session IDs internally. Each call to `client.query()` automatically continues the same session. Call `client.receive_response()` to iterate over the messages for the current query. Use the client as an async context manager so connection setup and teardown are handled for you, or call `connect()` and `disconnect()` manually.

This example runs two queries against the same `client`. The first asks the agent to analyze a module; the second asks it to refactor that module. Because both calls go through the same client instance, the second query has full context from the first without any explicit `resume` or session ID:

```

import asyncio
from claude_agent_sdk import (
    ClaudeSDKClient,
    ClaudeAgentOptions,
    AssistantMessage,
    ResultMessage,
    TextBlock,
)

def print_response(message):
    """Print only the human-readable parts of a message."""
    if isinstance(message, AssistantMessage):
        for block in message.content:
            if isinstance(block, TextBlock):
                print(block.text)
    elif isinstance(message, ResultMessage):
        cost = (
            f"${message.total_cost_usd:.4f}"
            if message.total_cost_usd is not None
            else "N/A"
        )
        print(f"[done: {message.subtype}, cost: {cost}]")

async def main():
    options = ClaudeAgentOptions(
        allowed_tools=["Read", "Edit", "Glob", "Grep"],
    )

    async with ClaudeSDKClient(options=options) as client:
        # First query: client captures the session ID internally
        await client.query("Analyze the auth module")
        async for message in client.receive_response():
            print_response(message)

        # Second query: automatically continues the same session
        await client.query("Now refactor it to use JWT")
        async for message in client.receive_response():
            print_response(message)

```

```
asyncio.run(main())
```

See the [Python SDK reference](#) for details on when to use `ClaudeSDKClient` vs the standalone `query()` function.

TypeScript: `continue: true`

The TypeScript SDK doesn't have a session-holding client object like Python's `ClaudeSDKClient`. Instead, pass `continue: true` on each subsequent `query()` call and the SDK picks up the most recent session in the current directory. No ID tracking required.

This example makes two separate `query()` calls. The first creates a fresh session; the second sets `continue: true`, which tells the SDK to find and resume the most recent session on disk. The agent has full context from the first call:

TypeScript

```
import { query } from "@anthropic-ai/claude-agent-sdk";

// First query: creates a new session
for await (const message of query({
  prompt: "Analyze the auth module",
  options: { allowedTools: ["Read", "Glob", "Grep"] }
})) {
  if (message.type === "result" && message.subtype === "success") {
    console.log(message.result);
  }
}

// Second query: continue: true resumes the most recent session
for await (const message of query({
  prompt: "Now refactor it to use JWT",
  options: {
    continue: true,
    allowedTools: ["Read", "Edit", "Write", "Glob", "Grep"]
  }
})) {
  if (message.type === "result" && message.subtype === "success") {
    console.log(message.result);
  }
}
```

❗ The experimental **V2 session API**, which provided `createSession()` with a `send / stream` pattern, was removed in TypeScript Agent SDK 0.3.142. Use the `query()` function and the session options described on this page instead.

Use session options with `query()`

Capture the session ID

Resume and fork require a session ID. Read it from the `session_id` field on the result message (`ResultMessage` in Python, `SDKResultMessage` in TypeScript), which is present on every result regardless of success or error. In TypeScript the ID is also available earlier as a direct field on the `init SystemMessage`; in Python it's nested inside `SystemMessage.data`.

```
import asyncio
from claude_agent_sdk import query, ClaudeAgentOptions, ResultMessage

async def main():
    session_id = None

    async for message in query(
        prompt="Analyze the auth module and suggest improvements",
        options=ClaudeAgentOptions(
            allowed_tools=["Read", "Glob", "Grep"],
        ),
    ):
        if isinstance(message, ResultMessage):
            session_id = message.session_id
            if message.subtype == "success":
                print(message.result)

    print(f"Session ID: {session_id}")
    return session_id

session_id = asyncio.run(main())
```

Resume by ID

Pass a session ID to `resume` to return to that specific session. The agent picks up with full context

from wherever the session left off. Common reasons to resume:

- **Follow up on a completed task.** The agent already analyzed something; now you want it to act on that analysis without re-reading files.
- **Recover from a limit.** The first run ended with `error_max_turns` or `error_max_budget_usd` (see [Handle the result](#)); resume with a higher limit.
- **Restart your process.** You captured the ID before shutdown and want to restore the conversation.

This example resumes the session from [Capture the session ID](#) with a follow-up prompt. Because you're resuming, the agent already has the prior analysis in context:

```
# Earlier session analyzed the code; now build on that analysis
async for message in query(
    prompt="Now implement the refactoring you suggested",
    options=ClaudeAgentOptions(
        resume=session_id,
        allowed_tools=["Read", "Edit", "Write", "Glob", "Grep"],
    ),
):
    if isinstance(message, ResultMessage) and message.subtype == "success":
        print(message.result)
```

You should see a response that builds on the earlier analysis instead of starting fresh. That confirms the agent resumed the session with its prior context intact.

💡 If a `resume` call returns a fresh session instead of the expected history, the most common cause is a mismatched `cwd`. Sessions are stored under `~/.claude/projects/<encoded-cwd>/*.jsonl`, or under `$CLAUDE_CONFIG_DIR/projects/<encoded-cwd>/*.jsonl` if you set the `CLAUDE_CONFIG_DIR` environment variable, where `<encoded-cwd>` is the absolute working directory with every non-alphanumeric character replaced by `-` (so `/Users/me/proj` becomes `-Users-me-proj`). If your resume call runs from a different directory, the SDK looks in the wrong place. The session file also needs to exist on the current machine.

To resume sessions across machines or in serverless environments, mirror transcripts to shared storage with a [SessionStore adapter](#).

Fork to explore alternatives

Forking creates a new session that starts with a copy of the original's history but diverges from that

point. The fork gets its own session ID; the original's ID and history stay unchanged. You end up with two independent sessions you can resume separately.

❗ Forking branches the conversation history, not the filesystem. If a forked agent edits files, those changes are real and visible to any session working in the same directory. To branch and revert file changes, use [file checkpointing](#).

This example builds on **Capture the session ID**: you've already analyzed an auth module in `session_id` and want to explore OAuth2 without losing the JWT-focused thread. The first block forks the session and captures the fork's ID (`forked_id`); the second block resumes the original `session_id` to continue down the JWT path. You now have two session IDs pointing at two separate histories:

```
# Fork: branch from session_id into a new session
forked_id = None
async for message in query(
    prompt="Instead of JWT, outline how OAuth2 would work for the auth
module",
    options=ClaudeAgentOptions(
        resume=session_id,
        fork_session=True,
        max_turns=5,
    ),
):
    if isinstance(message, ResultMessage):
        forked_id = message.session_id # The fork's ID, distinct from
session_id
        if message.subtype == "success":
            print(message.result)

print(f"Forked session: {forked_id}")

# Original session is untouched; resuming it continues the JWT thread
async for message in query(
    prompt="Continue with the JWT approach",
    options=ClaudeAgentOptions(resume=session_id),
):
    if isinstance(message, ResultMessage) and message.subtype == "success":
        print(message.result)
```

You should see that `forkedId` differs from the original session ID. Resuming the original session still

continues the JWT thread, which confirms the fork did not modify the original history.

Resume across hosts

Session files are local to the machine that created them. To resume a session on a different host (CI workers, ephemeral containers, serverless), you have two options:

- **Move the session file.** Persist `~/.claude/projects/<encoded-cwd>/<session-id>.jsonl` from the first run and restore it to the same path on the new host before calling `resume`. The `cwd` must match.
- **Don't rely on session resume.** Capture the results you need (analysis output, decisions, file diffs) as application state and pass them into a fresh session's prompt. This is often more robust than shipping transcript files around.

Both SDKs expose functions for enumerating sessions on disk and reading their messages:

`listSessions()` and `getSessionMessages()` in TypeScript, `list_sessions()` and `get_session_messages()` in Python. Use them to build custom session pickers, cleanup logic, or transcript viewers.

Both SDKs also expose functions for looking up and mutating individual sessions:

`get_session_info()`, `rename_session()`, and `tag_session()` in Python, and `getSessionInfo()`, `renameSession()`, and `tagSession()` in TypeScript. Use them to organize sessions by tag or give them human-readable titles.

Related resources

- **How the agent loop works**: Understand turns, messages, and context accumulation within a session
- **File checkpointing**: Snapshot and revert file changes the agent made within a session
- **Python `ClaudeAgentOptions`**: Full session option reference for Python
- **TypeScript `Options`**: Full session option reference for TypeScript